

Prompt:

8/17/2026, 9:49:12 PM

lanjut Tahap G

Response:

8/17/2026, 9:49:14 PM

Baik. **Tahap G adalah titik di mana seluruh arsitektur yang kita bangun mulai dipadatkan menjadi satu formulasi matematis yang benar-benar bisa dieksekusi oleh QAOA.**

Sampai Tahap F kita sudah punya banyak komponen:

market data → state → forecast → risk → candidate trades → execution → dynamic management.

Sekarang kita harus menjawab pertanyaan yang jauh lebih fundamental:

"Sebenarnya apa tepatnya yang sedang dioptimalkan oleh QAOA?"

Dan kita akan membangun jawabannya dari **objective finansial → binary decision → QUBO → Ising Hamiltonian → QAOA.**

Tahap G — Mathematical Formulation of the Trading Optimization Problem

Saya akan membaginya menjadi:

1. mendefinisikan universe aset,
2. mendefinisikan keputusan trading,
3. expected return,
4. covariance/risk,
5. market factors,
6. transaction cost,
7. turnover,
8. exposure constraints,
9. risk budget,
10. mengubah semuanya menjadi QUBO,
11. mapping QUBO → Ising,

12. membentuk cost Hamiltonian,
 13. membentuk mixer,
 14. QAOA,
 15. interpretasi output QAOA sebagai trade.
-

G.1. Universe aset

Misalkan pada waktu t kita memiliki:

$$N$$

aset.

Kita definisikan:

$$A = \{1, 2, \dots, N\}.$$

Contoh untuk crypto:

$$A = \{BTC, ETH, SOL, BNB, XRP, \dots\}.$$

Untuk saham:

$$A = \{AAPL, MSFT, NVDA, AMZN, \dots\}.$$

Tetapi sistem kita juga mempunyai **factor assets**.

Misalnya:

$$F = \{DXY, Gold, SPX, VIX\}.$$

Untuk crypto:

$$F = \{BTC, ETH, BTC.D, ETH/BTC\}.$$

G.2. Keputusan pertama: apakah aset dipilih?

Kita definisikan binary variable:

$$x_i \in \{0, 1\}$$



dengan:

$$x_i = \begin{cases} 1, & \text{aset } i \text{ diperdagangkan,} \\ 0, & \text{tidak.} \end{cases}$$

Jadi:

$$x = (x_1, x_2, \dots, x_N).$$

Contoh:

$$x = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}.$$

Artinya:

$$A_1, A_3$$

dipilih.

G.3. Tetapi trading bukan sekadar "dipilih/tidak"

Kita membutuhkan arah.

Karena:

$$BUY \not\equiv SELL.$$

Ada dua pendekatan.

Pendekatan A — dua binary variables

Definisikan:

$$b_i \in \{0, 1\}$$

untuk BUY,

dan:

$$s_i \in \{0, 1\}$$



untuk SELL.

Dengan constraint:

$$b_i + s_i \leq 1.$$

Pendekatan B — spin

Kita dapat menggunakan:

$$z_i \in \{-1, 0, +1\}.$$

dengan:

$$z_i = \begin{cases} +1, & \text{BUY} \\ 0, & \text{FLAT} \\ -1, & \text{SELL.} \end{cases}$$

Namun:

$$z_i \in \{-1, 0, +1\}$$

bukan binary.

Karena QUBO secara natural menggunakan:

$$x_i \in \{0, 1\},$$

saya menyarankan untuk **versi pertama menggunakan dua binary variables**.

G.4. Representasi BUY/SELL

Kita punya:

$$b_i, s_i \in \{0, 1\}.$$

Net direction:

$$d_i = b_i - s_i.$$

Maka:

$$d_i \in \{-1, 0, +1\}.$$

Contoh:

b_i	s_i	d_i
0	0	0
1	0	+1
0	1	-1
1	1	0*

Tetapi kombinasi:

$$b_i = s_i = 1$$

harus dilarang.

G.5. Constraint arah

Kita tambahkan:

$$b_i + s_i \leq 1.$$

Dalam QUBO kita bisa menulis penalti:

$$P_{\text{dir}} \sum_i b_i s_i.$$

Karena jika:

$$b_i = s_i = 1,$$

maka:

$$b_i s_i = 1$$

dan sistem menerima penalti.

G.6. Expected return

Sekarang kita membutuhkan:

$$\mu_i(t).$$

Ini bukan sekadar historical mean.

Kita definisikan:

$$\mu_t = E[r_{t+\Delta} | S_t].$$

Jadi:

$$\mu_t = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_N \end{bmatrix}.$$

Nilainya dapat berasal dari model forecasting Tahap B.

Misalnya:

$$\mu_i = \alpha_i + \beta_i^T F_t + \theta_i^T X_{i,t}.$$

G.7. Apa arti μ_i ?

Misalkan:

$$\mu_{BTC} = 0.002.$$

Artinya model memperkirakan:

$$E[r_{BTC,t+\Delta}] = 0.2\%.$$

Jika:

$$\mu_i < 0,$$

maka LONG tidak menarik.

Tetapi SHORT mungkin menarik.

Karena expected directional return adalah:

$$d_i \mu_i.$$

Jika:

$$d_i = -1$$

dan:

$$\mu_i = -0.002,$$

maka:

$$d_i \mu_i = (+0.002).$$

Jadi SHORT mendapatkan expected return positif.

G.8. Expected return objective

Kita ingin:

$$\max_d \sum_i d_i \mu_i.$$

Karena QUBO biasanya ditulis sebagai minimization, kita ubah menjadi:

$$C_{\text{return}} = - \sum_i \mu_i (b_i - s_i).$$

G.9. Risk

Sekarang kita memasukkan covariance.

$$\Sigma = E[(r - \mu)(r - \mu)^T].$$

Dengan:

$$\Sigma_{ij} = \text{Cov}(r_i, r_j).$$

G.10. Portfolio risk

Jika:

$$d = (d_1, \dots, d_N)^T,$$

maka:

$$\sigma_p^2 = d^T \Sigma d.$$

Ekspansi:

$$\sigma_p^2 = \sum_i \sum_j d_i \Sigma_{ij} d_j.$$

Karena:

$$d_i = b_i - s_i,$$

maka:

$$\sigma_p^2 = (b - s)^T \Sigma (b - s).$$

G.11. Ini adalah sumber coupling QUBO

Perhatikan:

$$d^T \Sigma d.$$

Ada:

$$d_i d_j.$$

Artinya keputusan memilih aset i bergantung pada apakah kita juga memilih aset j .

Inilah struktur:

combinatorial interaction.

Dan ini alasan mengapa QUBO/Ising menjadi relevan.

G.12. Objective dasar Markowitz

Kita dapat menggunakan:

$$C(d) = -\mu^T d + \lambda d^T \Sigma d.$$

Dengan:

$$\lambda > 0.$$

Semakin besar:

$$\lambda,$$

semakin risk-averse.

G.13. Tetapi trading kita tidak boleh berhenti di Markowitz

Kita ingin:

$$DXY, Gold, SPX, VIX, BTC.D, \dots$$

masuk ke model.

Ada dua cara.

Cara pertama:

factor sudah masuk ke:

$$\mu_t.$$

Tetapi itu belum cukup.

Kita juga ingin mengendalikan **factor exposure**.

G.14. Factor model

Kita definisikan:

$$r_i = \alpha_i + \beta_i^T f + \epsilon_i.$$

Dengan:

$$f = (f_1, \dots, f_K)^T.$$

Untuk aset i :

$$\beta_i = (\beta_{i1}, \dots, \beta_{iK}).$$

Kita susun:

$$B = \begin{bmatrix} \beta_{11} & \beta_{12} & \cdots & \beta_{1K} \\ \beta_{21} & \beta_{22} & \cdots & \beta_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{N1} & \beta_{N2} & \cdots & \beta_{NK} \end{bmatrix}.$$

G.15. Portfolio factor exposure

$$E_f = B^T d.$$

Untuk factor k :

$$E_k = \sum_i \beta_{ik} d_i.$$

G.16. Mengapa ini berguna?

Misalnya kita memiliki:

$$\beta_{DXY, BTC} = -0.7.$$

Jika kita LONG BTC:

$$d_{BTC} = +1,$$

maka:

$$E_{DXY} < 0.$$

Artinya portfolio kita memiliki negative DXY exposure.

Jika model macro mengatakan:

$$DXY \uparrow$$

dengan kuat, kita mungkin ingin mengurangi exposure tersebut.

G.17. Factor exposure constraint

Misalnya kita tidak ingin:

$$|E_{DXY}| > E_{\max}.$$

Secara matematis:

$$-E_{\max} \leq B_{DXY}^T d \leq E_{\max}.$$

Namun QUBO lebih mudah menggunakan squared penalty:

$$P_F (B_{DXY}^T d - E_{DXY}^{target})^2.$$

G.18. Multi-factor penalty

Untuk semua factor:

$$C_F = \sum_{k=1}^K P_k (\beta_k^T d - e_k^*)^2.$$

Di mana:

$$e_k^*$$

adalah target exposure.

Contoh:

$$e_{DXY}^* = 0.$$

Artinya kita ingin portfolio market-neutral terhadap DXY.

G.19. Ini menarik untuk strategi global

Misalnya kita trading:

$$AAPL, NVDA, AMD, TSLA.$$

Model menemukan semua expected return positif.

Naifnya:

$$LONG\ AAPL, NVDA, AMD, TSLA.$$

Tetapi semuanya mungkin mempunyai:

$$\beta_{SPX} > 0.$$

Maka portfolio sebenarnya hanya satu taruhan:

$$\text{US equity risk-on.}$$

Factor penalty mencegah optimizer menganggap keempatnya sebagai empat sumber alpha yang independen.

G.20. Risk decomposition

Covariance juga dapat ditulis melalui factor model:

$$\Sigma = BF B^T + D$$

dengan:

$$F = \text{factor covariance}$$

dan:

D = idiosyncratic covariance.

Maka:

$$d^T \Sigma d = d^T B F B^T d + d^T D d.$$

Sehingga:

$Risk = FactorRisk + IdiosyncraticRisk.$

Ini jauh lebih informatif daripada covariance matrix mentah.

G.21. Transaction cost

Sekarang kita masukkan biaya.

Untuk aset i :

$$c_i = fee_i + spread_i + slippage_i + impact_i.$$

Jika kita hanya mempertimbangkan apakah aset ditradingkan:

$$C_{TC} = \sum_i c_i (b_i + s_i).$$

G.22. Tetapi kita juga ingin turnover

Misalkan posisi sebelumnya:

$$d_i^{prev}.$$

Kita tidak ingin optimizer setiap periode berubah total.

Definisikan:

$$\Delta d_i = d_i - d_i^{prev}.$$

Transaction cost lebih tepat:

$$C_{TC} = \sum_i c_i |\Delta d_i|.$$

Masalahnya:

$$|\cdot|$$

tidak quadratic.

Untuk versi binary sederhana kita bisa mengaproksimasi dengan penalti switching.

Misalnya jika posisi sebelumnya LONG:

$$b_i^{prev} = 1,$$

maka pindah ke FLAT/SHORT diberi biaya.

G.23. Binary switching penalty

Salah satu bentuk sederhana:

$$C_{\text{switch}} = P_{\text{switch}} \sum_i (b_i - b_i^{prev})^2 + (s_i - s_i^{prev})^2.$$

Karena:

$$x^2 = x$$

untuk binary variable, ekspansi menjadi quadratic.

G.24. Kenapa turnover penting?

Tanpa turnover penalty:

$$\lambda_{\text{turnover}} = 0,$$

optimizer mungkin melakukan:

$$LONG \rightarrow SHORT \rightarrow LONG \rightarrow SHORT$$

setiap periode kecil.

Secara teori terlihat optimal.

Secara nyata:

$$TC \rightarrow \text{besar.}$$

Jadi:

alpha harus mengalahkan trading cost.

G.25. Cardinality constraint

Kita mungkin tidak ingin 50 aset sekaligus.

Misalnya maksimal:

$$K = 5.$$

Maka:

$$\sum_i (b_i + s_i) \leq K.$$

G.26. Mengubah cardinality menjadi penalty

Kita bisa menggunakan:

$$C_K = P_K \left(\sum_i (b_i + s_i) - K \right)^2.$$

Tetapi hati-hati.

Bentuk ini memberikan minimum ketika jumlah posisi tepat:

$$K,$$

bukan:

$$\leq K.$$

Jika kita benar-benar ingin "maksimal K ", formulasi QUBO perlu auxiliary variables atau bentuk penalty yang sesuai.

Untuk versi awal, justru kita dapat menggunakan:

$$\sum_i (b_i + s_i) = K.$$

Misalnya:

pilih tepat 5 kandidat trade.

G.27. Budget constraint

Misalkan setiap aset membutuhkan capital:

$$w_i.$$

Kita ingin:

$$\sum_i w_i (b_i + s_i) = W.$$

Penalty:

$$C_W = P_W \left(\sum_i w_i (b_i + s_i) - W \right)^2.$$

G.28. Risk budget

Kita ingin:

$$d^T \Sigma d \leq R_{\max}.$$

Ini merupakan constraint quadratic yang agak lebih sulit.

Untuk binary QUBO, ada dua pendekatan.

Pendekatan pertama

Masukkan risk langsung ke objective:

$$\lambda d^T \Sigma d.$$

Pendekatan kedua

Gunakan auxiliary/slack variables untuk mengubah inequality menjadi equality.

Untuk versi pertama, saya menyarankan pendekatan pertama.

G.29. Objective lengkap versi pertama

Sekarang kita sudah memiliki:

$$C = C_{\text{return}} + C_{\text{risk}} + C_{\text{factor}} + C_{\text{TC}} + C_{\text{cardinality}} + C_{\text{direction}} + C_{\text{switch}}.$$

Mari kita tulis semuanya.

G.30. Full binary formulation

Dengan:

$$d = b - s,$$

kita definisikan:

$$\begin{aligned} C(b, s) = & -\mu^T (b - s) \\ & + \lambda (b - s)^T \Sigma (b - s) \\ & + \sum_k P_k [\beta_k^T (b - s) - e_k^*]^2 \\ & + \sum_i c_i (b_i + s_i) \\ & + P_{\text{dir}} \sum_i b_i s_i \\ & + P_{\text{card}} [\sum_i (b_i + s_i) - K] \\ & + P_{\text{switch}} \sum_i [(b_i - b_i^{\text{prev}})^2 + (s_i - s_i^{\text{prev}})^2]. \end{aligned}$$

dengan:

$$b_i, s_i \in \{0, 1\}.$$

Inilah inti mathematical specification sistem kita.

G.31. Kenapa fungsi ini adalah QUBO?

Karena semua komponennya maksimal quadratic.

Bentuk umum:

$$C(x) = x^T Q x + c^T x + c_0.$$

dengan:

$$x = \begin{bmatrix} b_1 \\ \vdots \\ b_N \\ s_1 \\ \vdots \\ s_N \end{bmatrix}.$$

Karena:

$$x_i \in \{0, 1\},$$

maka ini adalah:

$$\boxed{\text{QUBO.}}$$

G.32. Sekarang kita turunkan matrix Q

Ini bagian penting.

Misalkan:

$$d = b - s.$$

Risk term:

$$d^T \Sigma d.$$

Ekspansi:

$$(b - s)^T \Sigma (b - s).$$

Maka:

$$= b^T \Sigma b - b^T \Sigma s - s^T \Sigma b + s^T \Sigma s.$$

Karena:

$$\Sigma = \Sigma^T,$$

maka:

$$b^T \Sigma s = s^T \Sigma b.$$

Sehingga:

$$d^T \Sigma d = b^T \Sigma b - 2b^T \Sigma s + s^T \Sigma s.$$

G.33. Matrix block structure

Maka risk matrix memiliki struktur:

$$Q_{\text{risk}} = \lambda \begin{bmatrix} \Sigma & -\Sigma \\ -\Sigma & \Sigma \end{bmatrix}.$$

Ini sangat elegan.

Bagian:

$$\Sigma$$

menghubungkan BUY-BUY.

Bagian:

$$-\Sigma$$

menghubungkan BUY-SELL.

G.34. Return term

Kita memiliki:

$$-\mu^T(b - s).$$

Sehingga:

$$-\mu^T b + \mu^T s.$$

Linear coefficient:

$$q_b = -\mu$$

dan:

$$q_s = +\mu.$$

G.35. Factor penalty

Untuk satu factor:

$$P_F(\beta^T d - e^*)^2.$$

Dengan:

$$d = b - s.$$

Definisikan:

$$a = \begin{bmatrix} \beta \\ -\beta \end{bmatrix}.$$

Maka:

$$\beta^T d = a^T x.$$

Sehingga:

$$P_F(a^T x - e^*)^2.$$

Ekspansi:

$$P_F[x^T a a^T x - 2e^* a^T x + (e^*)^2].$$

Jadi quadratic contribution:

$$Q_F = P_F a a^T.$$

G.36. Ini berarti factor exposure menciptakan coupling

Jika:

$$\beta_i \not\equiv 0$$

dan:

$$\beta_j \not\equiv 0,$$

maka:

$$\beta_i \beta_j$$

menghasilkan:

$$x_i x_j.$$

Jadi factor neutrality juga menghasilkan:

$$\text{quadratic interaction.}$$

G.37. Cardinality penalty

Misalkan:

$$y_i = b_i + s_i.$$

Kemudian:

$$P_K \left(\sum_i y_i - K \right)^2.$$

Ekspansi:

$$P_K \left[\sum_i y_i^2 + 2 \sum_{i < j} y_i y_j - 2K \sum_i y_i + K^2 \right].$$

Karena:

$$y_i^2 = y_i$$

jika y_i binary hanya bila y_i sendiri binary; tetapi:

$$y_i = b_i + s_i$$

dapat bernilai 2 jika constraint direction belum diberlakukan.

Lebih aman mengekspansi langsung b, s .

G.38. Direction constraint

$$P_{\text{dir}} \sum_i b_i s_i.$$

Mudah:

$$Q_{b_i s_i} += P_{\text{dir}}.$$

Jadi:

$$b_i = s_i = 1$$

menjadi mahal.

G.39. Switching penalty

Misalnya:

$$P_s(b_i - b_i^{prev})^2.$$

Karena:

$$b_i^2 = b_i,$$

maka:

$$= P_s[b_i - 2b_i b_i^{prev} + (b_i^{prev})^2].$$

Karena:

$$b_i^{prev} \in \{0, 1\},$$

maka:

$$q_i += P_s(1 - 2b_i^{prev}).$$

Jadi posisi sebelumnya mempengaruhi linear coefficient.

Ini berarti QUBO kita **dinamis**.

G.40. QUBO pada setiap waktu

Pada setiap:

$$t,$$

kita memiliki:

$$Q_t = Q(\mu_t, \Sigma_t, B_t, TC_t, d_{t-1}).$$

Jadi:

$$Q_{t_1} \boxplus Q_{t_2}$$

secara umum.

Ini merupakan konsekuensi langsung dari dynamic optimization yang kita bahas di Tahap E.

G.41. Sekarang kita masuk ke Ising

QAOA bekerja secara natural dengan:

$$Z_i$$

yang memiliki eigenvalue:

$$z_i \in \{-1, +1\}.$$

Sedangkan QUBO menggunakan:

$$x_i \in \{0, 1\}.$$

Gunakan mapping:

$$x_i = \frac{1 - z_i}{2}.$$

atau:

$$z_i = 1 - 2x_i.$$

G.42. Mengubah linear term

Misalkan:

$$c_i x_i.$$

Dengan:

$$x_i = \frac{1 - z_i}{2},$$

maka:

$$c_i x_i = \frac{c_i}{2} - \frac{c_i}{2} z_i.$$

Jadi:

$$c_i x_i \rightarrow \frac{c_i}{2} - \frac{c_i}{2} z_i.$$

G.43. Quadratic term

Misalkan:

$$Q_{ij} x_i x_j.$$

Substitusi:

$$x_i x_j = \frac{(1 - z_i)(1 - z_j)}{4}.$$

Maka:

$$= \frac{1}{4}(1 - z_i - z_j + z_i z_j).$$

Jadi:

$$Q_{ij} x_i x_j \rightarrow \frac{Q_{ij}}{4}(1 - z_i - z_j + z_i z_j).$$

G.44. Maka Hamiltonian

Secara umum:

$$C(x) = x^T Q x + c^T x + c_0.$$

menjadi:

$$H_C = C_0 I + \sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j.$$

Konstanta:

$$C_0$$

tidak memengaruhi argmin.

G.45. Koefisien Ising

Tergantung konvensi apakah Q menyimpan faktor dua pada off-diagonal, tetapi secara konseptual:

$$J_{ij} \propto Q_{ij}.$$

Sedangkan:

$$h_i$$

berasal dari kombinasi:

- linear QUBO coefficient,
- quadratic couplings yang melibatkan i .

Untuk implementasi nanti kita harus memilih **satu konvensi matrix Q** dan tidak boleh mencampurnya.

Ini penting sekali karena kesalahan faktor 2 adalah salah satu bug paling umum ketika mengubah QUBO ke Ising.

G.46. Cost Hamiltonian

Kita akhirnya memperoleh:

$$\hat{H}_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Jika computational basis:

$$|z\rangle = |z_1 z_2 \cdots z_n\rangle,$$

maka:

$$\hat{H}_C |z\rangle = C(z) |z\rangle.$$

Artinya:

$$\text{eigenvalue Hamiltonian} = \text{cost trading decision.}$$

Ini adalah koneksi langsung antara quantum mechanics dan optimization problem kita.

G.47. Interpretasi fisiknya

Misalkan:

$$h_i < 0.$$

Maka sistem cenderung memilih:

$$Z_i = +1$$

atau, tergantung mapping yang kita gunakan:

$$x_i = 0.$$

Sebaliknya jika:

$$h_i > 0,$$

kecenderungannya berbeda.

Tetapi keputusan akhir tidak hanya ditentukan oleh:

$$h_i.$$

Ada:

$$J_{ij}.$$

Jadi aset tidak berdiri sendiri.

G.48. Coupling J_{ij}

Misalnya:

$$J_{ij} > 0.$$

Term:

$$J_{ij}Z_iZ_j$$

cenderung meminimalkan energi ketika:

$$Z_iZ_j = -1.$$

Artinya konfigurasi anti-aligned.

Sedangkan:

$$J_{ij} < 0$$

cenderung:

$$Z_iZ_j = +1.$$

Ini memberikan interpretasi Ising.

Dalam konteks trading, coupling dapat berasal dari:

- covariance,
- factor exposure,
- cardinality,
- transaction cost,
- constraints.

Jadi:

J_{ij} bukan hanya correlation.

Ia merupakan gabungan seluruh interaksi dalam objective.

G.49. Cost Hamiltonian bukan portfolio Hamiltonian murni

Ini penting.

Kita jangan mengatakan:

J_{ij} = correlation.

Lebih tepat:

$$J_{ij} = f(\Sigma_{ij}, \beta_i, \beta_j, constraints, costs, \dots).$$

G.50. QAOA

Sekarang kita mempunyai:

$$\hat{H}_C.$$

QAOA menggunakan mixer:

$$\hat{H}_M = \sum_i X_i.$$

Untuk unconstrained binary optimization.

G.51. Initial state

Kita mulai dengan:

$$|+\rangle^{\otimes n}.$$

Dengan:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

Sehingga:

$$|\psi_0\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle.$$

Artinya seluruh kandidat trading decision memiliki amplitude awal yang sama.

G.52. QAOA layer

Untuk:

$$p$$

layers:

$$|\psi(\gamma, \beta)\rangle = \prod_{l=1}^p e^{-i\beta_l H_M} e^{-i\gamma_l H_C} |\psi_0\rangle.$$

Urutannya:

$$U_C(\gamma) = e^{-i\gamma H_C}$$

kemudian:

$$U_M(\beta) = e^{-i\beta H_M}.$$

G.53. Mengapa cost Hamiltonian tidak mengubah probabilitas langsung?

Karena:

$$U_C$$

di computational basis hanya memberikan phase:

$$|x\rangle \rightarrow e^{-i\gamma C(x)} |x\rangle.$$

Jadi:

$$\text{cost layer} \rightarrow \text{phase encoding.}$$

Kemudian mixer:

$$U_M$$

mengubah amplitudes antar state.

Interferensi antara keduanya menyebabkan state dengan cost yang lebih baik memperoleh probability lebih besar setelah parameter QAOA dioptimalkan.

G.54. Objective QAOA

Kita ingin:

$$E(\gamma, \beta) = \langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle$$

seminimal mungkin.

Jadi:

$$(\gamma^*, \beta^*) = \arg \min_{\gamma, \beta} E(\gamma, \beta).$$

Perhatikan:

ini adalah optimization kedua.

G.55. Dua optimization yang sering tertukar

Optimization pertama:

$$x^* = \arg \min_x C(x)$$

adalah **trading combinatorial optimization**.

Optimization kedua:

$$(\gamma^*, \beta^*) = \arg \min_{\gamma, \beta} \langle H_C \rangle$$

adalah **variational parameter optimization**.

Jadi QAOA adalah nested optimization:

$$\text{classical optimizer} \rightarrow (\gamma, \beta) \rightarrow \text{quantum circuit} \rightarrow \langle H_C \rangle.$$

G.56. Setelah QAOA

Setelah parameter optimal:

$$(\gamma^*, \beta^*),$$

kita memperoleh:

$$|\psi^*\rangle.$$

Kemudian kita lakukan measurement:

$$M$$

shots.

Contoh:

011001

muncul:

2,430

kali.

State dengan frequency tertinggi menjadi kandidat:

$$x^*.$$

G.57. Contoh interpretasi

Misalkan kita memiliki 4 aset:

$$\{BTC, ETH, SOL, BNB\}.$$

Dengan 2 bits per asset:

$$(b_1, b_2, b_3, b_4)$$

dan:

$$(s_1, s_2, s_3, s_4).$$

Total:

$$n = 8$$

qubits.

Misalnya hasil QAOA:

$$b = (1, 0, 1, 0)$$

$$s = (0, 0, 0, 1).$$

Maka:

$$d = b - s$$

menjadi:

$$d = (1, 0, 1, -1).$$

Interpretasi:

LONG BTC

FLAT ETH

LONG SOL

SHORT BNB.

G.58. Kemudian trade constructor

QAOA tidak langsung menghasilkan:

$$SL = \dots$$

atau:

$$TP = \dots$$

Ia menghasilkan:

discrete trading allocation/selection.

Kemudian Trade Construction Engine menggunakan:

$$S_t$$

untuk menentukan:

$$Entry_i, SL_i, TP_i, q_i.$$

G.59. Ini pembagian kerja yang sangat penting

QAOA

Menjawab:

Apa yang harus diperdagangkan?

Classical risk/position optimizer

Menjawab:

Seberapa besar?

Position manager

Menjawab:

Kapan harus mempertahankan, memodifikasi, atau keluar?

Execution engine

Menjawab:

Bagaimana order tersebut dieksekusi?

Dengan demikian kita tidak memaksa QAOA menjadi "super algorithm" yang melakukan segala sesuatu.

G.60. Position sizing

Misalkan QAOA memilih:

$$LONG\ BTC.$$

Trade constructor menghitung:

$$q_i = \frac{R_i}{E[Loss_i|SL_i]}.$$

Jika:

$$R_i = \$1$$

dan:

$$E[Loss_i|SL_i] = \$0.8$$

maka:

$$q_i = 1.25.$$

G.61. Tetapi kita perlu portfolio risk adjustment

Jika ada tiga posisi:

$$BTC, ETH, SOL$$

masing-masing:

$$R_i = \$1,$$

tidak berarti total risk:

$$\$3.$$

Karena:

$$\text{Cov}(BTC, ETH) > 0.$$

Jadi kita menghitung:

$$\sigma_p^2 = q^T \Sigma q.$$

Kemudian melakukan scaling:

$$q_{\text{scaled}} = cq$$

dengan:

$$c = \min \left(1, \sqrt{\frac{V_{\max}}{q^T \Sigma q}} \right).$$

G.62. Ini memberi dua lapis risk control

Lapisan 1

QUBO:

risk-aware selection.

Lapisan 2

Position sizing:

risk-aware allocation.

Ini jauh lebih aman daripada hanya mengandalkan optimizer.

G.63. Sekarang masalah penting: discrete vs continuous

Kita sebenarnya mempunyai:

$$w_i \in \mathbb{R}$$

untuk position size.

Sedangkan QAOA bekerja dengan:

$$x_i \in \{0, 1\}.$$

Jangan dipaksakan semuanya menjadi binary.

Saya menyarankan:

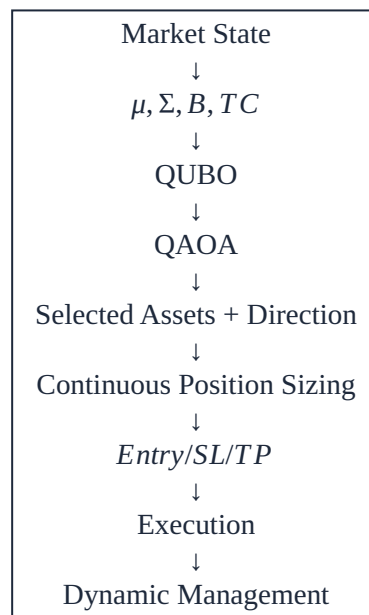
$QAOA = \text{asset/direction selection}$

dan:

$Classicaloptimizer = \text{continuous sizing.}$

Ini hybrid architecture yang lebih natural.

G.64. Maka final architecture kita



G.65. Tetapi ada masalah besar yang harus kita selesaikan sebelum coding

Jumlah qubit.

Jika:

$$N = 100$$

dan kita menggunakan:

$$2N$$

binary variables:

$$n = 200$$

qubits.

QAOA simulasi statevector membutuhkan:

$$2^{200}$$

amplitudes.

Ini mustahil secara brute-force.

Maka kita harus menggunakan **candidate universe reduction**.

G.66. Candidate selection

Jangan berikan 5,000 saham kepada QAOA.

Gunakan classical pre-filter:

$$A_{\text{candidate}} \subset A.$$

Misalnya:

$$5000 \rightarrow 100 \rightarrow 20.$$

Baru:

$$20$$

masuk QUBO.

G.67. Bagaimana memilih candidate?

Misalnya:

$$Score_i = f(\mu_i, liquidity_i, volatility_i, factor\ compatibility_i).$$

Ambil:

$$Top - K.$$

Misalnya:

$$K = 12.$$

Maka:

$$2K = 24$$

qubits untuk direction encoding.

Ini jauh lebih realistis untuk eksperimen awal.

G.68. Tetapi candidate pre-filter harus causal

Jangan menggunakan:

$$future\ return$$

untuk memilih candidate.

Harus:

$$Score_i(t) = f(D_{\leq t}).$$

Ini kembali ke prinsip Tahap F.

G.69. Alternative encoding

Kita juga bisa menggunakan satu binary variable:

$$x_i \in \{0, 1\}$$

dan direction ditentukan oleh sign:

$$\text{sign}(\mu_i).$$

Misalnya:

$$\mu_i > 0 \Rightarrow \text{LONG},$$

$$\mu_i < 0 \Rightarrow \text{SHORT}.$$

Kemudian QAOA hanya memilih:

$$x_i.$$

Ini membutuhkan:

$$N$$

qubits, bukan:

$$2N.$$

Untuk versi awal saya sangat menyarankan **encoding ini sebagai baseline.**

G.70. Formulasi simplified QUBO

Kita definisikan:

$$d_i = s_i x_i$$

dengan:

$$s_i = \text{sign}(\mu_i).$$

Maka:

$$d_i \in \{-1, 0, +1\}.$$

Tetapi karena sign sudah ditentukan:

$$d_i = \text{sign}(\mu_i)x_i.$$

Objective:

$$C(x) = - \sum_i |\mu_i| x_i + \lambda \sum_{ij} s_i s_j \Sigma_{ij} x_i x_j + \text{penalties}.$$

Sekarang hanya:

$$N$$

qubits.

G.71. Apa kelemahannya?

Kita tidak bisa secara independen memilih:

$$LONG$$

atau:

$$SHORT.$$

Direction ditentukan oleh forecast.

Namun ini justru dapat menjadi **baseline QAOA pertama yang sangat bersih**.

Kemudian kita bandingkan dengan:

$$2N$$

direction-aware formulation.

G.72. Saya menyarankan eksperimen bertahap

Model G1

Asset selection only.

N qubits.

Direction berasal dari forecast.

Model G2

Asset + direction.

$2N$ qubits.

Model G3

Asset + direction + cardinality.

Model G4

Asset + direction + factor exposure.

Model G5

Full formulation:

return + risk + factor + cost + turnover + constraints.

Dengan demikian kita dapat melihat kontribusi setiap komponen.

G.73. Ini sangat penting untuk eksperimen ilmiah

Kalau kita langsung memasukkan:

20

constraint sekaligus dan performanya buruk, kita tidak tahu penyebabnya.

Dengan ablation study:

$G1 \rightarrow G2 \rightarrow G3 \rightarrow G4 \rightarrow G5,$

kita dapat melihat:

$\Delta Performance.$

G.74. Contoh ablation

Misalnya hasil:

Model	Sharpe	MDD	Turnover
G1	1.12	-18%	4.2
G2	1.25	-16%	4.8
G3	1.39	-13%	3.1
G4	1.51	-11%	2.8
G5	1.57	-9%	1.9

Maka kita dapat mengatakan:

factor exposure dan transaction-cost-aware optimization memberikan kontribusi terhadap peningkatan risk-adjusted performance.

Itu jauh lebih kuat secara ilmiah.

G.75. Dan sekarang kita dapat melihat hubungan dengan skripsimu

Ada hubungan yang sangat langsung dengan apa yang telah kamu pelajari sebelumnya.

Kamu sudah bekerja dengan:

Markowitz → QUBO → Ising → VQE/QAOA.

Dalam proyek ini kita memperluasnya menjadi:

Markowitz + factor model + transaction cost + turnover + dynamic state

kemudian:

→ QUBO → Ising → QAOA.

Jadi ini bukan algoritma quantum yang ditempelkan ke trading.

Kita mulai dari **financial optimization problem**, lalu menunjukkan bahwa problem tersebut memiliki struktur combinatorial yang dapat dipetakan ke QUBO.

G.76. Bahkan ada hubungan langsung dengan Hamiltonian Ising yang sudah kamu pelajari

Kita sebelumnya menulis:

$$\hat{H} = - \sum_i h_i Z_i - \sum_{i < j} J_{ij} Z_i Z_j.$$

Sekarang kita dapat menginterpretasikan:

$$h_i$$

sebagai efek individual keputusan trading, sedangkan:

$$J_{ij}$$

sebagai interaksi antar keputusan.

Interaksi tersebut dapat berasal dari:

covariance + factor exposure + constraints + transaction costs.

Jadi secara fisik:

market portfolio ↔ spin system

dengan energy landscape yang merepresentasikan objective finansial.

G.77. Tetapi jangan membuat klaim terlalu besar

Kita **tidak** boleh mengatakan:

"Karena pasar adalah sistem Ising."

Itu tidak benar.

Yang benar:

Kita memetakan **masalah optimasi diskrit yang berasal dari portfolio/trading problem** ke bentuk QUBO/Ising.

Jadi Ising adalah **representasi matematis optimization problem**, bukan klaim bahwa pasar secara literal merupakan material magnetik.

Ini penting secara akademik.

G.78. Tahap G dalam satu persamaan

Kalau seluruh pembahasan ini harus diringkas menjadi satu formula, saya akan menggunakan:

$$\begin{aligned} \min_{b,s} \quad & -\mu_t^T(b-s) + \lambda(b-s)^T \Sigma_t(b-s) \\ & + \sum_k P_k [\beta_{k,t}^T(b-s) - e_{k,t}^*]^2 \\ & + \sum_i c_{i,t}(b_i + s_i) \\ & + P_{\text{switch}} \|b - b_{t-1}\|_2^2 + P_{\text{switch}} \|s - s_{t-1}\|_2^2 \\ & + P_{\text{dir}} \sum_i b_i s_i \\ & + P_K [\sum_i (b_i + s_i) - K] \\ \text{s.t.} \quad & b_i, s_i \in \{0, 1\}. \end{aligned}$$

Kemudian:

$$\min_{x \in \{0,1\}^{2N}} x^T Q_t x + c_t^T x + c_0$$

kemudian:

$$x_i = \frac{1 - Z_i}{2}$$

kemudian:

$$H_C(t) = \sum_i h_i(t) Z_i + \sum_{i < j} J_{ij}(t) Z_i Z_j.$$

Dan akhirnya:

$$|\psi_t(\gamma, \beta)\rangle = \prod_{l=1}^p e^{-i\beta_l H_M} e^{-i\gamma_l H_C(t)} |+\rangle^{\otimes n}.$$

Dengan:

$$(\gamma^*, \beta^*) = \arg \min_{\gamma, \beta} \langle H_C(t) \rangle.$$

G.79. Namun ada satu hal yang sangat penting untuk Tahap H

Kita sekarang sudah mempunyai **mathematical formulation**.

Tetapi masih ada satu pertanyaan besar:

Bagaimana kita menentukan μ_t , Σ_t , dan β_t secara benar dari data?

Karena seluruh QUBO bergantung pada:

$$\mu_t, \quad \Sigma_t, \quad B_t.$$

Kalau estimasinya buruk, QAOA bisa memberikan **solusi optimal untuk QUBO yang salah**.

Dan ini merupakan jebakan yang sangat penting:

$$\text{Optimization quality} \not\equiv \text{forecast quality}.$$

QAOA bisa menemukan:

$$x^*$$

dengan sangat baik, tetapi jika:

$$\mu_t$$

salah, maka:

$$x^*$$

tetap bisa menjadi trade yang buruk.

G.80. Maka Tahap H seharusnya bukan langsung coding QAOA

Menurut saya, tahap berikutnya justru harus kita bangun sebagai:

Tahap H — Forecasting & State Estimation Engine

Di sana kita akan menurunkan secara matematis:

$$\mu_t$$

dari:

- historical returns,
- momentum,
- volatility,
- volume,
- BTC/ETH,
- DXY,
- gold,
- SPX/VIX,
- cross-asset correlation,
- order-book imbalance,

kemudian membangun:

$$\Sigma_t$$

secara **rolling / exponentially weighted / shrinkage**, dan:

$$B_t$$

untuk factor exposure.

Yang paling menarik, kita juga bisa membuat **regime-dependent estimator**, sehingga:

$$\mu_t, \Sigma_t, B_t = f(S_t).$$

Dengan demikian QUBO kita benar-benar menjadi:

$$Q_t = f(\text{market state at } t)$$

dan bukan sekadar QUBO statis yang dihitung sekali dari historical data.

Baru setelah Tahap H selesai, kita mempunyai seluruh bahan yang dibutuhkan untuk membuat implementasi QAOA yang serius: bukan hanya circuit-nya, tetapi juga **bagaimana angka-angka yang masuk ke circuit tersebut diperoleh secara causal dari pasar**.